

StyleAI — AI Wishlist

Style Preview

MVP that reduces wishlist purchase uncertainty. Shoppers upload a photo; Groq analyzes it with the product image and metadata and returns structured style-confidence guidance. This is AI Style Preview — not generated virtual try-on.

Live frontend: <https://wishlist-1ick.vercel.app>

GitHub: <https://github.com/Somu639/Wishlist>

Date: 4 September 2026 · Audience: Product, design, engineering

1. Problem and north star

Wishlisted fashion sits unused because shoppers cannot tell whether a piece will style with what they already wear. Fake scarcity and discount badges do not answer that question. StyleAI gives an honest, photo-based style recommendation so the shopper can add to bag or leave the item with more confidence.

North star: 30-day wishlist → purchase conversion. Experiment: conversion among users who use AI Style Preview vs those who do not.

2. Product flow

Home / Women PLP → Wishlist (heart) → AI Style on a card → upload shopper photo → Get My AI Recommendation → score, verdict, why it works, style-it-with, occasions, what to consider → Move to Bag / Place Order.

- Storefront opens on Women clothing with 8 seeded products (dress, kurti, saree, co-ord, jeans, shirt, blazer, anarkali).
- Myntra-style header: Men, Women, Kids, Home & Living, Beauty, Studio; search; Profile / Wishlist / Bag badges.
- Wishlist intelligence tiles: Ready to Buy, Need Reconsideration, Style Uncertainty, Possibly Outdated, Alternative Available.
- AI modal is labelled AI Style Preview. It never claims the shopper is wearing the product.
- Bag shows MRP, discount, convenience fee, free delivery, and Place Order (demo checkout).

3. What the AI returns

Field	Rule
overall_score	Integer 0–100 for style compatibility, not attractiveness
verdict	3–6 words, or exactly “Unable to confidently analyze the image.”
why_it_works	Exactly 3 short styling reasons
styling_suggestions	Exactly 3 wearable combinations

Field	Rule
best_occasions	Exactly 3 India / urban occasions from metadata
matching_colors / accessories	Exactly 3 each
confidence_gaps	Exactly 2 honest caveats
purchase_recommendation	1–2 sentences, not pushy, no fit guarantee
size_guidance	Metadata sizes only — a photo cannot confirm size
disclaimer	AI-generated style guidance; not a guarantee of fit or appearance.

Forbidden: body-shaming, attractiveness ranking, race / caste / skin-tone ranking, age or health inference, exact measurements, guaranteed fit, sexualised language, hallucinated fabric / brand / price.

4. Catalog (seeded wishlist)

ID	Product	Brand	Price	MRP	Off
PROD001	Floral Wrap Midi Dress	HERE&NOW	■1,299	■2,599	50%
PROD002	Bandhani Print Cotton Kurti	BIBA	■999	■1,799	44%
PROD003	Kanjivaram Silk Saree	Kalki Fashion	■8,499	■12,999	35%
PROD004	Oversized Linen Co-ord Set	MANGO	■3,499	■5,999	42%
PROD005	High-Rise Straight Fit Jeans	Levis	■2,799	■3,999	30%
PROD006	Relaxed Fit Printed Oversized Shirt	H&M	■1,499	■2,499	40%
PROD007	Structured Power Blazer	AND	■3,999	■6,499	38%
PROD008	Embroidered Anarkali Suit Set	W for Woman	■4,299	■6,999	39%

5. Architecture

Three runtimes share the same catalog and Groq prompt. Streamlit Community Cloud cannot host the Express REST API, so Vercel is self-contained for the public demo.

Surface	Role	URL / command
Vercel frontend	React storefront + wishlist + bag + AI modal	https://wishlist-1ick.vercel.app
Vercel /api/style	Serverless Groq vision analysis	POST JSON: product_id + image_base64
Express backend	SQLite wishlist/cart + /api/analyze-style	http://localhost:5000
Streamlit backend	Python Groq UI (same catalog + intelligence)	http://localhost:8501 · streamlit_backend/app.py
GitHub	Source of truth	https://github.com/Somu639/Wishlist

Stack

- Frontend: React 18, Vite, Tailwind. Colors: #ff3f6c pink, #282c3f ink, #ff905a discount, #03a685 success.
- Express: Node 22+, node:sqlite, multer (memory), sharp, Groq SDK, helmet, CORS for *.vercel.app.
- Vercel function: api/style.js — no native deps; retries JSON-mode / product-URL failures; hides reasoning.
- Streamlit: streamlit_backend/app.py + groq + Pillow. Secrets: GROQ_API_KEY, GROQ_VISION_MODEL.
- Vision model default: qwen/qwen3.6-27b. Photos resized (768–1024px JPEG) and never written to disk.

6. Express API

Method	Path	Purpose
GET	/health	Health + AI capabilities
GET	/api/wishlist	Wishlist + intelligence summary
POST	/api/wishlist	Add { product_id }
DELETE	/api/wishlist/:id	Remove from wishlist
GET / POST / DELETE	/api/cart	Bag lines; source=ai_preview tracks conversion
POST	/api/analyze-style	Multipart userPhoto + product_id
POST	/api/analyze-style/save	Persist analysis JSON
POST / GET	/api/analytics/*	Events and summary
POST	/api/style (Vercel)	JSON image_base64 + product_id

7. How to run locally

Express + React

1. In backend/.env set a real GROQ_API_KEY (gsk_...) and GROQ_VISION_MODEL=qwen/qwen3.6-27b.
 2. cd backend && npm install && npm run dev → <http://localhost:5000>
 3. cd frontend && npm install && npm run dev → <http://localhost:5173>
- Vite proxies /api to Express. Wishlist seeds 8 items into SQLite on first boot.

Streamlit Python backend

```
pip install -r streamlit_backend/requirements.txt
streamlit run streamlit_backend/app.py
```

Put GROQ_API_KEY in .streamlit/secrets.toml (gitignored) or the environment. UI: <http://localhost:8501>

8. Deploy

Vercel (already live)

- Repo root is the Vercel project. vercel.json builds frontend/ and exposes api/style.js (maxDuration 60s).
- Do not put requirements.txt at repo root — Vercel then looks for a Python entrypoint and the build fails.
- Set GROQ_API_KEY (and optional GROQ_VISION_MODEL) in Vercel → Settings → Environment Variables, then Redeploy.
- Wishlist/bag use localStorage when Express is absent, so the storefront is never empty.
- Hard-refresh <https://wishlist-1ick.vercel.app> after each deploy (Ctrl+Shift+R).

Streamlit Community Cloud

- share.streamlit.io → Somu639/Wishlist → main → main file streamlit_backend/app.py.
- Secrets TOML: GROQ_API_KEY and GROQ_VISION_MODEL. Packages come from streamlit_backend/requirements.txt.
- Streamlit is a UI, not a REST host. The Vercel React app cannot call it as /api.

9. Analytics events

wishlist_view, wishlist_product_selected, photo_uploaded, try_on_started, ai_analysis_completed, ai_analysis_failed, style_recommendation_viewed, add_to_cart, add_to_cart_after_ai, conversion_after_ai, try_another_product. Payloads may include product_id, product_category, ai_score, analysis_latency_ms.

10. Privacy and product rules

- Shopper photos stay in memory, are resized, sent to Groq, and discarded.
- UI copy says AI Style Preview — never try-on, overlay, or “this definitely fits you.”
- Size guidance uses available_sizes from metadata only.
- If the photo is dark, cropped, or covered: score 20–40 and verdict “Unable to confidently analyze the image.”
- No fake scarcity, countdown timers, or invented discounts.
- Footer marks this as an internal product prototype.

11. Known limits

- backend/.env in local clones may still hold the placeholder your_groq_api_key_here — paste a real gsk_key.
- Groq developer-plan rate limits return HTTP 429; the UI asks the shopper to wait and retry.
- Men / Kids / Home & Living / Beauty / Studio aisles are empty in this 8-SKU women's catalog.
- Place Order is a demo toast — no payment is collected.
- SQLite on serverless should use /tmp if Express is hosted on Render (render.yaml sets SQLITE_PATH).

12. Repo layout

backend/src/ Express API, Groq service, SQLite
frontend/src/ Storefront, Wishlist, Style Preview, Bag
frontend/src/data/ Catalog + localStorage store + intelligence
api/style.js Vercel serverless Groq endpoint
streamlit_backend/ Streamlit app, prompts, Groq, requirements.txt
docs/ This PDF

End of brief. Live demo: <https://wishlist-1ick.vercel.app>